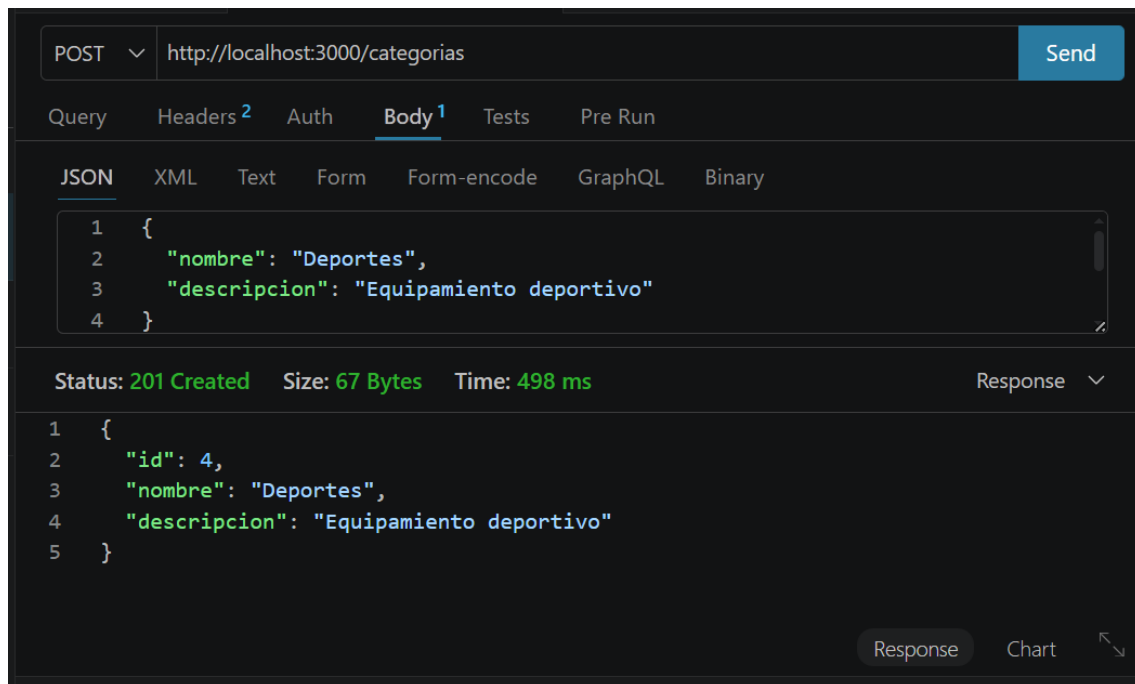


PRACTICA # 2

Ejercicio 1

Crea un endpoint POST /categorias que permita registrar una nueva categoría enviando nombre y descripción en el body de la petición.



Ejercicio 2

Crea un endpoint GET /categorias que devuelva todas las categorías registradas en la base de datos.

GET

▼

http://localhost:3000/categorias

Send

Query

Headers2

Auth

Body1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

1

{

2

"nombre": "Deportes",

3

"descripcion": "Equipamiento deportivo"

4

}

Status: 200 OK

Size: 612 Bytes

Time: 12 ms

Response

▼

1

[

2

{

3

"id": 1,

4

"nombre": "Electrónica",

5

"descripcion": "Dispositivos electrónicos y gadgets",

6

"createdAt": "2026-06-02T15:15:29.000Z",

7

"updatedAt": "2026-06-02T15:15:29.000Z"

8

},

9

{

10

"id": 2,

11

"nombre": "Oficina",

12

"descripcion": "Material y accesorios de oficina",

13

"createdAt": "2026-06-02T15:15:29.000Z",

14

"updatedAt": "2026-06-02T15:15:29.000Z"

15

}

16

]

Copy

Response

Chart

16

{

17

"id": 3,

18

"nombre": "Deportes",

19

"descripcion": "Equipamiento deportivo",

20

"createdAt": "2026-06-02T15:28:32.000Z",

21

"updatedAt": "2026-06-02T15:28:32.000Z"

22

},

23

{

24

"id": 4,

25

"nombre": "Deportes",

26

"descripcion": "Equipamiento deportivo",

27

"createdAt": "2026-06-02T22:50:26.000Z",

28

"updatedAt": "2026-06-02T22:50:26.000Z"

29

}

Copy

Response

Chart

☐	 Editar	 Copiar	 Borrar	1	Electrónica	Dispositivos electrónicos y gadgets	2026-06-02 11:15:29	2026-06-02 11:15:29
☐	 Editar	 Copiar	 Borrar	2	Oficina	Material y accesorios de oficina	2026-06-02 11:15:29	2026-06-02 11:15:29
☐	 Editar	 Copiar	 Borrar	3	Deportes	Equipamiento deportivo	2026-06-02 11:28:32	2026-06-02 11:28:32
☐	 Editar	 Copiar	 Borrar	4	Deportes	Equipamiento deportivo	2026-06-02 18:50:26	2026-06-02 18:50:26

Ejercicio 3

Crea un endpoint GET /categorias/:id que devuelva la categoría con el ID especificado y además, incluya todos los productos que pertenecen a esa categoría.

GET ▼ http://localhost:3000/categorias/1 Send

Query Headers ² Auth Body ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "nombre": "Deportes",
3   "descripcion": "Equipamiento deportivo"
4 }
```

Status: 200 OK Size: 178 Bytes Time: 10 ms Response ▼

```
1 {
2   "id": 1,
3   "nombre": "Electrónica",
4   "descripcion": "Dispositivos electrónicos y gadgets",
5   "createdAt": "2026-06-02T15:15:29.000Z",
6   "updatedAt": "2026-06-02T15:15:29.000Z",
7   "productos": []
8 }
```

Copy

GET ▼ http://localhost:3000/categorias/3 Send

Query Headers ² Auth Body ¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "nombre": "Deportes",
3   "descripcion": "Equipamiento deportivo"
4 }
```

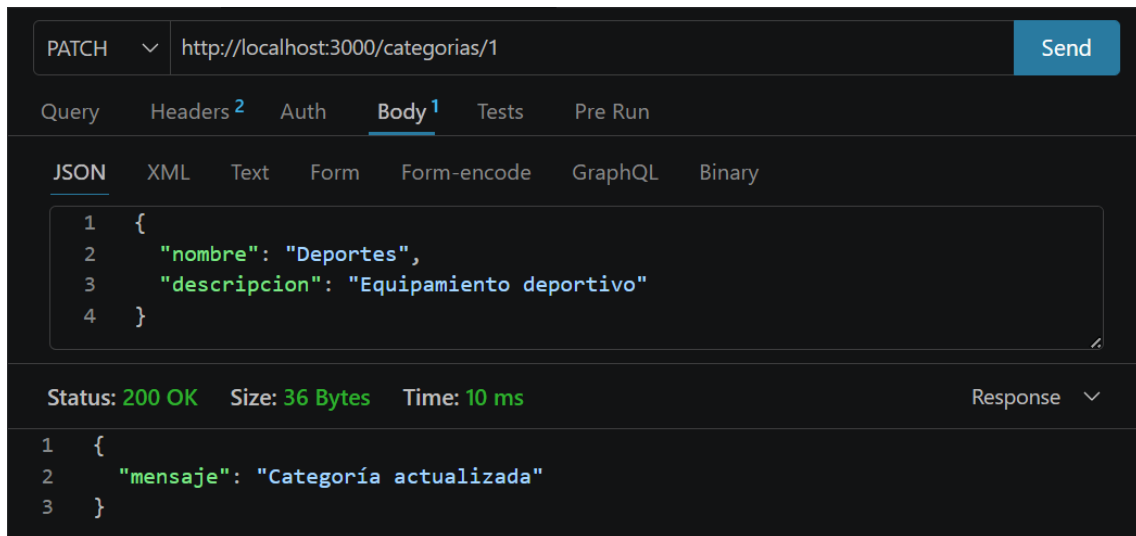
Status: 200 OK Size: 160 Bytes Time: 8 ms Response ▼

```
1 {
2   "id": 3,
3   "nombre": "Deportes",
4   "descripcion": "Equipamiento deportivo",
5   "createdAt": "2026-06-02T15:28:32.000Z",
6   "updatedAt": "2026-06-02T15:28:32.000Z",
7   "productos": []
8 }
```

Copy

Ejercicio 4

Crea un endpoint PATCH /categorias/:id que permita actualizar todos los datos de la categoría con el ID especificado.



Ejercicio 5

Crea un endpoint DELETE /categorias/:id que elimine la categoría indicada y, al mismo tiempo, elimine automáticamente todos los productos que pertenecen a esa categoría.

DELETE

http://localhost:3000/categorias/2

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

1

{

2

"nombre": "Deportes",

3

"descripcion": "Equipamiento deportivo"

4

}

Status: 200 OK

Size: 51 Bytes

Time: 8 ms

Response

1

{

2

"mensaje": "Categoría y sus productos eliminados"

3

}

Copy

☐	 Editar	 Copiar	 Borrar	1	Deportes	Equipamiento deportivo	2026-06-02 11:15:29	2026-06-02 19:04:07
☐	 Editar	 Copiar	 Borrar	3	Deportes	Equipamiento deportivo	2026-06-02 11:28:32	2026-06-02 11:28:32
☐	 Editar	 Copiar	 Borrar	4	Deportes	Equipamiento deportivo	2026-06-02 18:50:26	2026-06-02 18:50:26

Soluciones Código

Ejercicio 1

Archivo: routes/categorias.js

```
const express = require('express');
const router = express.Router();
const categoriaController = require('../controllers/categoriaController');

router.post('/', categoriaController.crearCategoria);

module.exports = router;
```

Archivo: controllers/categoriaController.js

```
const db = require('../db');

/
exports.crearCategoria = async (req, res) => {
  const { nombre, descripcion } = req.body;

  if (!nombre) {
    return res.status(400).json({ error: 'El nombre es obligatorio' });
  }

  try {
    const [result] = await db.query(
      'INSERT INTO categorias (nombre, descripcion) VALUES (?, ?)',
      [nombre, descripcion || null]
    );
    res.status(201).json({ id: result.insertId, nombre, descripcion });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}
```

```
}  
};
```

Ejercicio 2

Archivo: routes/categorias.js

```
router.get('/', categoriaController.obtenerCategorias);
```

Archivo: controllers/categoriaController.js

```
exports.obtenerCategorias = async (req, res) => {  
  try {  
    const [rows] = await db.query('SELECT * FROM categorias');  
    res.json(rows);  
  } catch (error) {  
    res.status(500).json({ error: error.message });  
  }  
};
```

Ejercicio 3

Archivo: routes/categorias.js

```
router.get('/:id', categoriaController.obtenerCategoriaConProductos);
```

Archivo: controllers/categoriaController.js

```
exports.obtenerCategoriaConProductos = async (req, res) => {  
  const { id } = req.params;  
  
  try {  
    const [categoria] = await db.query('SELECT * FROM categorias WHERE  
id = ?', [id]);  
    if (categoria.length === 0) {  
      return res.status(404).json({ error: 'Categoría no encontrada' });  
    }  
  
    const [productos] = await db.query('SELECT * FROM productos WHERE  
categoria_id = ?', [id]);
```

```

    res.json({
      ...categoria[0],
      productos
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

```

Ejercicio 4

Archivo: routes/categorias.js

```
router.patch('/:id', categoriaController.actualizarCategoria);
```

Archivo: controllers/categoriaController.js

```

exports.actualizarCategoria = async (req, res) => {
  const { id } = req.params;
  const { nombre, descripcion } = req.body;

  try {
    const [existing] = await db.query('SELECT * FROM categorias WHERE id = ?', [id]);

    if (existing.length === 0) {
      return res.status(404).json({ error: 'Categoría no encontrada' });
    }

    const nuevoNombre = nombre !== undefined ? nombre :
existing[0].nombre;

    const nuevaDesc = descripcion !== undefined ? descripcion :
existing[0].descripcion;

    await db.query(

```



```
'UPDATE categorias SET nombre = ?, descripcion = ?, updatedAt =  
NOW() WHERE id = ?',
```

```
[nuevoNombre, nuevaDesc, id]
```

```
);
```

```
res.json({ mensaje: 'Categoría actualizada' });
```

```
} catch (error) {
```

```
res.status(500).json({ error: error.message });
```

```
}
```

```
};
```

Ejercicio 5

Archivo: routes/categorias.js

```
router.delete('/:id', categoriaController.eliminarCategoria);
```

Archivo: controllers/categoriaController.js

```
exports.eliminarCategoria = async (req, res) => {
```

```
const { id } = req.params;
```

```
try {
```

```
const [result] = await db.query('DELETE FROM categorias WHERE id = ?',  
[id]);
```

```
if (result.affectedRows === 0) {
```

```
return res.status(404).json({ error: 'Categoría no encontrada' });
```

```
}
```

```
res.json({ mensaje: 'Categoría y sus productos eliminados' });
```

```
} catch (error) {
```

```
res.status(500).json({ error: error.message });
```

```
}
```

```
};
```